Software Requirements Specification

for

Automated Timetable Generation System

Version 1.0

Prepared by

Group Name: Team 4

Alekya Duvvuri	<student 1>	se23ucse014@mahindrauniversity.edu.in
Amit Verma	<student 2>	se23ucse020@mahindrauniversity.edu.in
Bobba Dilip Reddy	<student 3>	se23ucse038@mahindrauniversity.edu.in
Dhruv Nandigam	<student 4>	se23ucse055@mahindrauniversity.edu.in
Jayanth Reddy Yerabolu	<student 5>	se23ucse080@mahindrauniversity.edu.in
Rahul Kumar Singh	<student 6>	se23ucse150@mahindrauniversity.edu.in
Tallam Venkata Naga Sai Srujan	<student 7>	se23ucse171@mahindrauniversity.edu.in
Atharv Garg	<student 8>	se23ucse223@mahindrauniversity.edu.in

Instructor: Prof. Gopala Krishna Koneru

Course: Software Engineering [CS3201]

Lab Section: *IT2, First Floor Lab*

Teaching Assistant: *Prof. Murali Krishna Bukkasamudram*

Date: 09-03-2026

CONTENTS II
REVISIONS II
1 INTRODUCTION 1
1.1 Document Purpose 1
1.2 Product Scope 1
1.3 Intended Audience and Document Overview 1
1.4 Definitions, Acronyms and Abbreviations 1
1.5 Document Conventions 1
1.6 References and Acknowledgments 2
2 OVERALL DESCRIPTION 2
2.1 Product Overview 2
2.2 Product Functionality 3
2.3 Design and Implementation Constraints 3
2.4 Assumptions and Dependencies 3
3 SPECIFIC REQUIREMENTS 4
3.1 External Interface Requirements 4
3.2 Functional Requirements 4
3.3 Use Case Model 5
4 OTHER NON-FUNCTIONAL REQUIREMENTS 6
4.1 Performance Requirements 6
4.2 Safety and Security Requirements 6
4.3 Software Quality Attributes 6
5 OTHER REQUIREMENTS 7
APPENDIX A – DATA DICTIONARY 8
APPENDIX B - GROUP LOG 9

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0 Draft	All members	Initial SRS Document	8 Mar 2026

1 Introduction

This document is the Software Requirements Specification (SRS) for the Automated Timetable Generation System (ATGS). The system uses a Genetic Algorithm (GA) to automatically produce conflict-free academic timetables for educational institutions, replacing the tedious and error-prone manual scheduling process. This section introduces the purpose, scope, intended audience, and conventions used throughout the document.

1.1 Document Purpose

This SRS describes the requirements for the **Automated Timetable Generation System**, a software solution designed to automatically generate academic timetables for educational institutions using optimization techniques such as Genetic Algorithms.

The purpose of this document is to clearly define the functional and non-functional requirements of the system so that developers, instructors, and stakeholders have a common understanding of how the system should behave. This document serves as a reference for system design, implementation, testing, and future maintenance. It outlines the expected system features, constraints, interfaces, and overall architecture needed to successfully develop the project.

This SRS primarily focuses on the core components of the system including data ingestion, authentication and role management, preference submission, timetable generation using GA, and timetable visualization.

1.2 Product Scope

The Automated Timetable Generation System is a web-based academic scheduling platform. Administrators upload course, faculty, student, room, and time-slot data; faculty members submit their preferred teaching slots; and the backend GA engine produces a conflict-free, optimised timetable. The final schedule is displayed through role-specific views: students see their section timetable, faculty see their personal teaching schedule, and management gets a bird's-eye view of all allocations.

The primary benefits are:

- Reduce the time and manual effort required for timetable creation.
- Minimize scheduling conflicts related to classrooms, instructors, and student groups.
- Incorporate faculty preferences as soft constraints.
- Provide role-based access for administrators, professors, and students.
- Provide clear and accessible timetable views for different user roles.

1.3 Intended Audience and Document Overview

This document is intended for multiple audiences involved in the development and evaluation of the Automated Timetable Generation System.

The primary audience includes the **development team**, who will use this document as a reference for implementing system functionality. It is also intended for the **course instructor and teaching assistants**, who act as stakeholders and evaluators of the project. Additionally, this document may be useful for **future developers or maintainers** who need to understand the design and requirements of the system.

The document is structured as follows:

• **Section 1 – Introduction:** Provides an overview of the document and the project.

• **Section 2 – Overall Description:** Describes the system context, functionality, constraints, and assumptions.

• **Section 3 – Specific Requirements:** Lists detailed functional requirements and use case specifications.

• **Section 4 – Non-Functional Requirements:** Defines performance, security, and quality attributes of the system.

• **Section 5 – Other Requirements:** Covers additional requirements such as database or legal considerations.

• **Appendices:** Include supporting information such as the data dictionary and group activity log.

Readers interested in system functionality should primarily refer to Section 3, while system design considerations are covered in Section 2 and Section 4.

1.4 Definitions, Acronyms and Abbreviations

Term / Acronym	Definition
Admin	Administrative user responsible for uploading data and generating timetables
ATGS	Automated Timetable Generation System; the software being specified in this document.
Chromosome	Representation of a candidate timetable solution in the genetic algorithm
Fitness Function	The function that scores each candidate timetable based on constraint satisfaction and preference coverage.
GA	Genetic Algorithm. The optimisation algorithm at the core of the scheduling engine.
Hard Constraint	A scheduling rule that must never be violated (e.g., no two classes in the same room at the same time).
Population	A group of candidate timetable solutions used in genetic algorithm evolution
RBAC	Role-Based Access Control; a security model that restricts system access based on user roles.
Soft Constraint	A scheduling preference that should be satisfied where possible but may be relaxed (e.g., faculty preferred slots).
SRS	Software Requirements Specification; this document.

1.5 Document Conventions

This document follows IEEE 830-1998 formatting conventions.

The following conventions are used throughout the document:

- Section headings and subsections follow the numbering format defined in the template.
- **Font:** Arial, 11 pt for body text, with bold formatting used for headings and section titles.
- The word **"shall"** indicates a mandatory requirement, while **"should"** indicates a recommended but non-mandatory requirement.
- Important technical terms such as Genetic Algorithm, Hard Constraints, and Soft Constraints are defined in Section 1.4 (Definitions, Acronyms, and Abbreviations).
- Diagrams, such as use case diagrams and system overview diagrams, are included where necessary to illustrate system structure, components, and interactions.
- *Italicized text* is used for comments, notes, or clarifications that are not formal system requirements.

All text is written using a clear and consistent format to ensure readability and maintainability of the document.

1.6 References and Acknowledgments

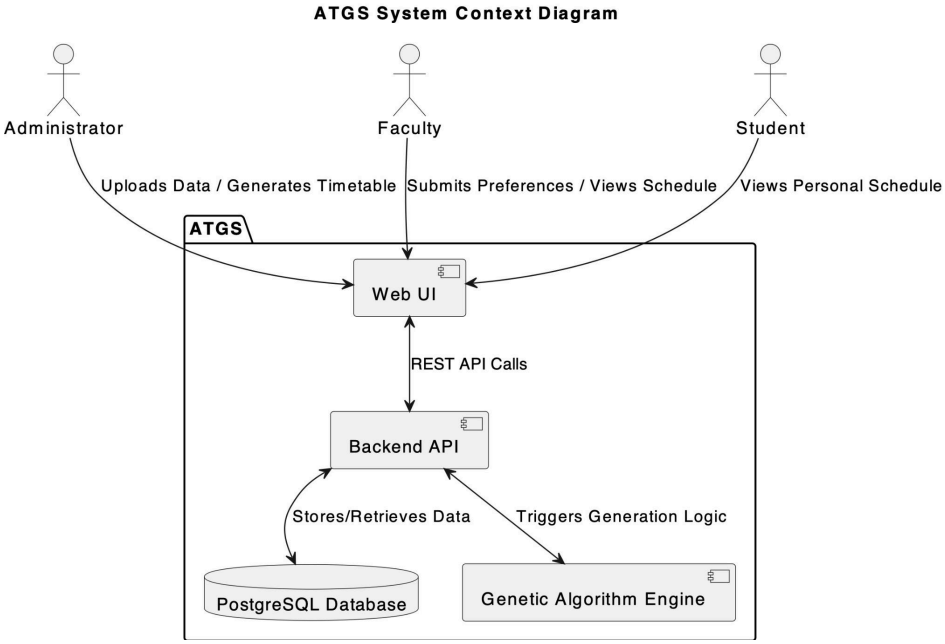
The following documents and resources were referenced during the preparation of this Software Requirements Specification:

- **Statement of Work - Automated Timetable Generation System**. Defines the project scope, objectives, milestones, and system deliverables.
- **IEEE Software Requirements Specification Template.** Template used for structuring this SRS document.
- **Goldberg, D. E. (1989). Genetic Algorithms in Search, Optimization, and Machine Learning.** Reference for the optimization technique used for timetable generation.
- **Sommerville, I. (2016). Software Engineering (10th Edition).** Reference for software engineering principles and system design practices.
- **Unified Modeling Language (UML) Documentation**. Reference for system modeling diagrams such as use case diagrams and system architecture diagrams.
- **Course Materials – Software Engineering Course (Mahindra University).** Guidelines and resources provided by the instructor for preparing SRS and UML models.
- **Official Documentation for Proposed Technologies (React, Node.js, PostgreSQL, etc.)** Referenced for implementation guidelines and best practices during system development.

2 Overall Description

2.1 Product Overview

The Automated Timetable Generation System (ATGS) is a standalone, web-based application designed to modernize academic scheduling. Traditionally, educational institutions rely on manual processes that are error-prone and labor-intensive; this product serves as a sophisticated replacement using optimization techniques. As a self-contained system, it manages the entire lifecycle of timetable creation—from the ingestion of raw institutional data to the final visualization of conflict-free schedules



2.2 Product Functionality

- **Authentication and RBAC:** Secure login and access control for Administrators, Faculty, and Students.
- **Data Ingestion:** Administrative tools to upload structured data (as CSV/Excel) for courses, rooms, and enrollments.
- **Preference Management:** A dedicated interface for faculty to submit teaching slot preferences.
- **Automated Scheduling:** Genetic Algorithm engine that evolves a population of solutions to find a conflict free timetable.
- **Constraint Validation:** Real time checking of hard constraints (e.g., no double-booked rooms or instructors).
- **Schedule Visualization:** Interactive weekly views tailored to the specific role of the logged-in user.
- **Export Capabilities:** Functionality to download the generated results in PDF and Excel formats.

2.3 Design and Implementation Constraints

- **Modeling and Design Standards:** The system must be designed using the **COMET**. All architectural and behavioral representations must utilize the **UML** for consistency and maintainability.
- **Hardware Interface Constraints:** The application must be optimized for cross-platform access, supporting client devices ranging from high-resolution desktop monitors to smartphones.
- **Algorithmic Complexity:** The core scheduling engine is constrained to use a **Genetic Algorithm**.
- **Language and Technology Stack:** The backend logic shall be implemented in **Python** to leverage existing optimization and data processing libraries. The database is restricted to **PostgreSQL** to handle complex relational queries for student enrollments and room allocations.
- **Security and Privacy:** The system must enforce **Role-Based Access Control (RBAC)**. All user passwords must be hashed using **bcrypt**, and no plain-text credentials may be stored or logged. All client-server communication must be encrypted via **HTTPS**.
- **Regulatory Compliance:** Documentation must adhere to the **IEEE 830-1998** standard.

2.4 Assumptions and Dependencies

- **Browser Compatibility:** The system assumes users are accessing the interface via modern web browsers (e.g., Chrome, Firefox).
- **Data Integrity:** It is assumed that all data uploaded via CSV/Excel follows the predefined system schema accurately.
- **Algorithm Convergence:** It is assumed that a feasible solution exists within the provided constraints for the Genetic Algorithm.
- **Computational and External Libraries:** The system depends on open-source libraries for PDF/Excel processing (e.g., ReportLab, Pandas) and high-performance numerical libraries like **NumPy** to handle large-scale matrix operations for Genetic Algorithm evolution and fitness scoring.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The Automatic Timetable Generation System provides a **web-based graphical user interface (GUI)** accessible through standard web browsers. The system supports three types of users: **Administrator, Faculty, and Students**.

The major user interface components are described below.

1. Login Interface

The system provides a login page where users authenticate using their credentials. After successful authentication, users are redirected to their respective dashboards based on their role.

2. Administrator Interface

The administrator interface allows the administrator to upload all required data and trigger timetable generation. The interface includes the following components:

- **Data Upload Interface**
  Allows uploading datasets such as:
  - Faculty information
  - Student information
  - Courses
  - Student enrollment
  - Infrastructure (classrooms/labs)
  - Time-slot definitions
- Data is uploaded through structured files (e.g., CSV or Excel).
- **Timetable Generation Interface**
  Provides a control for the administrator to initiate automatic timetable generation after all required data has been uploaded.
- **Timetable Display Interface**
  Displays the generated timetable in a tabular format organized by days and time slots.
- **Export Interface**
  Allows downloading the generated timetable in **Excel and PDF formats**.

3. Faculty Interface

Faculty members can log in and perform the following actions:

- Submit **preferred teaching time-slots**
- View their assigned timetable after generation

Faculty preferences may or may not be respected by the scheduling algorithm depending on constraint satisfaction.

4. Student Interface

Students can log in to:

- View their generated timetable based on enrolled courses.

The timetable is presented in a **weekly schedule format** organized by days and time-slots.

3.1.2 Hardware Interfaces

The Automatic Timetable Generation System operates as a **web-based application** and interacts with the following hardware components:

Device	Description
Client Devices	Computers, laptops, tablets, or smartphones used by users to access the system via a web browser.
Application Server	Hosts the backend application implemented in Python that processes scheduling logic and timetable generation.
Database Server	Stores system data including uploaded datasets, preferences, and generated timetables.
Network Infrastructure	Enables communication between client devices and the application server through internet or institutional networks.

3.1.3 Software Interfaces

The system interacts with the following software components:

Software Component	Description
Web Browser	Provides the interface through which users interact with the system.
Python Backend Application	Implements the timetable generation logic and constraint processing.
Database Management System	Stores uploaded data including faculty, students, courses, enrollments, infrastructure, and generated timetables.

File Processing Module	Processes uploaded datasets such as CSV or Excel files.
Export Module	Generates timetable files in Excel and PDF formats.

The backend application processes uploaded data and generates feasible timetables using scheduling algorithms while attempting to satisfy defined constraints and faculty preferences.

3.2 Functional Requirements

3.2.1 F1: User Authentication

The system shall allow administrators, faculty members, and students to log in using valid credentials.

3.2.2 F2: Role-Based Access Control

The system shall restrict access to system functionalities based on user roles (Admin, Faculty, Student).

3.2.3 F3: Data Upload

The system shall allow the administrator to upload datasets including faculty information, student information, courses, enrollments, infrastructure, and time-slots.

3.2.4 F4: Data Validation

The system shall validate uploaded data to ensure completeness and correctness before timetable generation.

3.2.5 F5: Faculty Preference Submission

The system shall allow faculty members to submit their preferred teaching time-slots.

3.2.6 F6: Preference Storage

The system shall store faculty time-slot preferences for use during timetable generation.

3.2.7 F7: Timetable Generation Trigger

The system shall allow the administrator to trigger automatic timetable generation after required data has been uploaded.

3.2.8 F8: Automatic Timetable Generation

The system shall automatically generate a timetable using the provided input data.

3.2.9 F9: Constraint Satisfaction

The system shall ensure that generated timetables satisfy system constraints such as (and are not limited to):

- No faculty assigned to multiple classes at the same time
- No classroom double booking
- No student schedule conflicts

3.2.10 F10: Preference Consideration

The system shall attempt to incorporate faculty preferred time-slots during timetable generation whenever possible. These preferences will be treated as soft constraints.

3.2.11 F11: Timetable Storage

The system shall store the generated timetable in the database.

3.2.12 F12: Timetable Viewing

The system shall allow faculty and students to view their generated timetable.

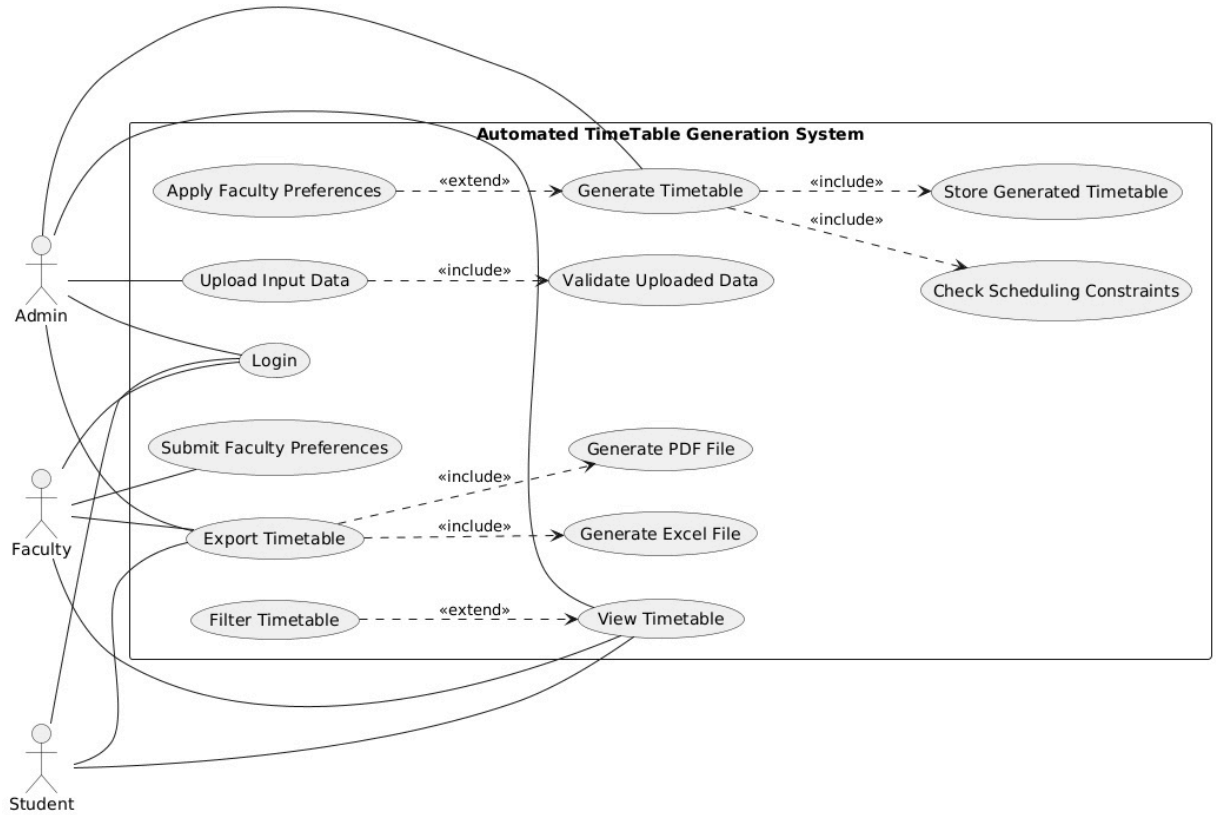
3.2.13 F13: Timetable Export

The system shall allow users to export timetables in Excel and PDF formats.

3.2.14 F14: Timetable Filtering

The system shall allow users to view timetables filtered by student, faculty, or course.

3.3 Use Case Model



3.3.1 U1 – User Login

Author: Team Member
Purpose: Authenticate users before granting access to the system.
Requirements Traceability: F1, F2
Priority: High

Preconditions

- User must have valid credentials.

Postconditions

- User is redirected to their dashboard.

Actors

- Administrator
- Faculty

- Student

Basic Flow

1. User opens login page.
2. User enters username and password.
3. System verifies credentials.
4. User is granted access to the dashboard.

Alternative Flow

- Invalid credentials → error message displayed.

3.3.2 U2 – Upload Input Data

Purpose: Allow the administrator to upload required datasets for timetable generation.
Requirements Traceability: F3, F4
Actors: Administrator
Priority: High

Preconditions

- Administrator must be logged in.

Basic Flow

1. Admin opens data upload interface.
2. Admin selects dataset file (faculty, students, courses, etc.).
3. System validates the file format.
4. System stores the uploaded data.

Alternative Flow

- If file format is invalid → system displays error.

3.3.3 U3 – Submit Faculty Preferences

Purpose: Allow faculty members to submit preferred time-slots.
Requirements Traceability: F5, F6
Actors: Faculty

Basic Flow

1. Faculty logs into the system.
2. Faculty opens preference submission interface.
3. Faculty selects preferred time-slots.
4. System stores the preferences.

3.3.4 U4 – Generate Timetable

Purpose: Automatically generate timetable based on input data and constraints.
Requirements Traceability: F7, F8, F9, F10
Actors: Administrator
Priority: High

Preconditions

- Required data must be uploaded.

Basic Flow

1. Admin triggers timetable generation.
2. System retrieves uploaded data and faculty preferences.
3. System executes timetable generation algorithm.
4. System validates constraints.
5. Generated timetable is stored.

Alternative Flow

- If constraints cannot be satisfied, system reports generation failure.

3.3.5 U5 – View Timetable

Purpose: Allow users to view generated timetable.
Requirements Traceability: F12
Actors: Student, Faculty, Administrator

Basic Flow

1. User logs in.
2. User selects timetable view.
3. System displays timetable relevant to the user.

3.3.6 U6 – Export Timetable

Purpose: Allow users to download timetable files.
Requirements Traceability: F13

Actors

- Administrator
- Faculty
- Student

Basic Flow

1. User selects export option.

2. User selects format (Excel or PDF).
3. System generates file.
4. File is downloaded.

4 Other Non-functional Requirements

4.1 Performance Requirements

- The system should generate a complete timetable within a reasonable time for typical academic data sizes.
- All REST API endpoints, except the timetable generation trigger, should respond quickly under normal system load.
- The timetable viewer page should load and display the timetable without noticeable delay on a standard internet connection.
- CSV imports of up to 1,000 records shall complete validation and preview rendering within 10 seconds.
- OTP delivery emails shall arrive within 60 seconds of the signup request.

4.2 Safety and Security Requirements

- All user passwords shall be hashed using bcrypt before storage. Plain-text passwords shall never be stored or logged.
- All communication between client and server must use HTTPS.
- API endpoints shall enforce RBAC (Role-Based Access Control). Accessing data outside a user's role (e.g., a Student calling Admin endpoints) will cause an error.
- Login attempts should be rate limited to prevent brute-force attacks.
- All user inputs shall be sanitised and validated server-side to prevent SQL injection and XSS attacks.
- The OTP for email verification shall be valid for 10 minutes and expire after a single use.
- Database operations should use transactions to prevent partial data updates.

4.3 Software Quality Attributes

- The system should maintain high availability during active use. If timetable generation fails, the system should save the best available solution instead of crashing.
- Error messages shall be human-readable and actionable (e.g., 'Room LH-101 is already fully booked for this time slot' rather than 'Constraint violation').
- The scheduling algorithm should be designed so it can be updated in the future without changing the rest of the system.
- The system should support adding or removing new constraints or roles without major changes.
- The entire application shall be deployed using Vercel. This ensures the project can be demonstrated on any machine without environment setup hassles.

5 Other Requirements

*<This section is **Optional**. Define any other requirements not covered elsewhere in the SRS. This might include database requirements, internationalization requirements, legal requirements, reuse objectives for the project, and so on. Add any new sections that are pertinent to the project.>*

Appendix A – Data Dictionary

<Data dictionary is used to track all the different variables, states and functional requirements that you described in your document. Make sure to include the complete list of all constants, state variables (and their possible states), inputs and outputs in a table. In the table, include the description of these items as well as all related operations and requirements.>

Appendix B - Group Log

<Please include here all the minutes from your group meetings, your group activities, and any other relevant information that will assist in determining the effort put forth to produce this document>